Funded by
the European Union

Reproducibility of results and its importance

Introduction to snakemake

FORGENOM II

Simone Ciccolella

BIAS
Bioinformatica e Algoritmica Sperimentale

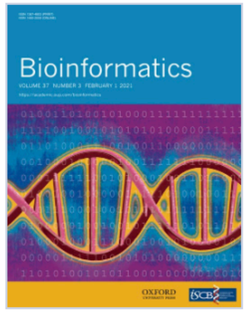
UNIVERSITÀ DEGLI STUDI DI MILANO
BICOCCA

Let’s look at the literature

OXFORD ACADEMIC | Journals | Books

Bioinformatics

Issues | Advance articles | Submit ▾ | Alerts | About ▾ | Bioinformatics



Volume 37, Issue 3
February 2021

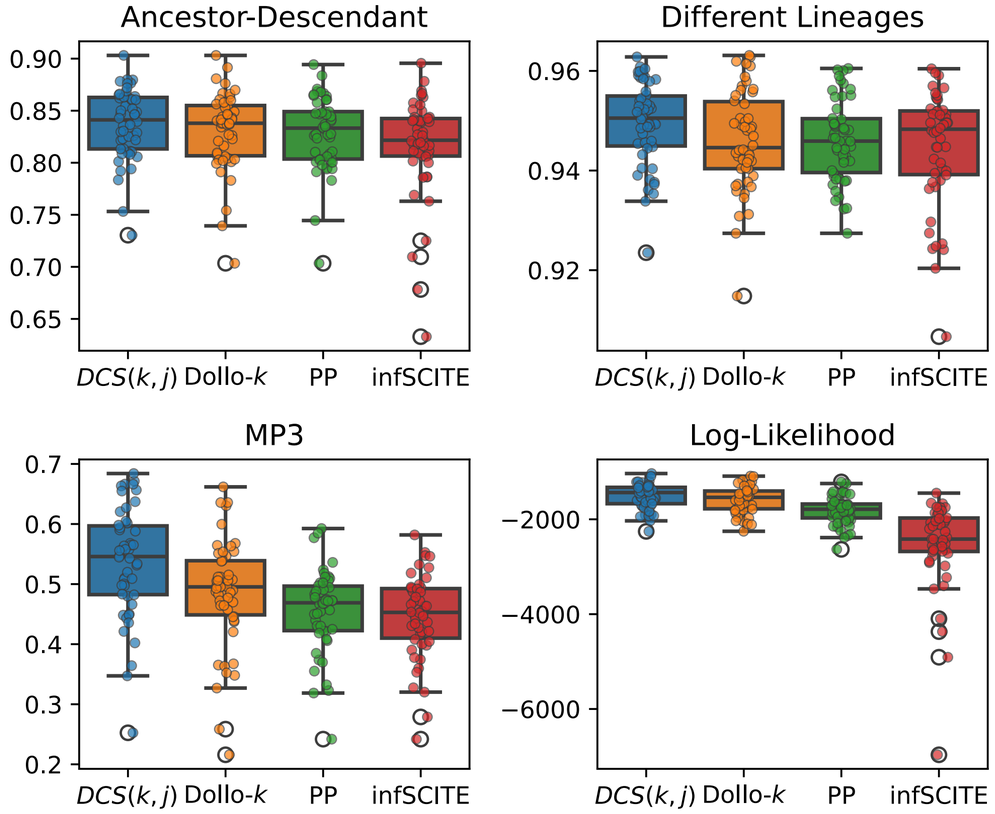
JOURNAL ARTICLE

Inferring cancer progression from Single-Cell Sequencing while allowing mutation losses

Simone Ciccolella , Camir Ricketts , Mauricio Soto Gomez , Murray Patterson , Dana Silverbush , Paola Bonizzoni , Iman Hajirasouliha , Gianluca Della Vedova

Bioinformatics, Volume 37, Issue 3, February 2021, Pages 326–333,
https://doi.org/10.1093/bioinformatics/btaa722
Published: 17 August 2020 **Article history** ▾

Let’s look at the literature



	Recurrences			Losses		
	P	R	F	P	R	F
$DCS(k,j)$	0. 69	0.48	0.56	0.67	0.73	0.70
Dollo-k	-	-	-	0.59	0.63	0.61

Experiment #	$DCS(k,j)$	Dollo-k	PP	infSCITE
1	2955 s	3019 s	2517 s	39183 s
2	3421 s	5006 s	2642 s	20583 s
3	3974 s	3440 s	2541 s	19208 s
4	4737 s	3645 s	2794 s	22688 s
5	4263 s	4460 s	3772 s	24362 s

Results

Availability and implementation

The SASC tool is open source and available at https://github.com/sciccolella/sasc.

Results

sasc Public

Pin | Unwatch 2 | Fork 1 | Star 3

master | 2 Branches | 4 Tags | Go to file t | Add file | Code

sciccolella add new implementation cf81bd3 · 4 months ago 2 Commits

.gitignore	first draft	4 months ago
2bfpbwt.c	add new implementation	4 months ago
Makefile	add new implementation	4 months ago
bf-pbwt.c	first draft	4 months ago

About

No description, website, or topics provided.

Activity

1 star

2 watching

1 fork

Report repository

Releases

No releases published

Create a new release

Packages

No packages published

Publish your first package

Contributors 3

Results

sasc Public

Pin | Unwatch 2 | Fork 1 | Star 3

master | 2 Branches | 4 Tags | Go to file t | Add file | Code | About

sciccolella add new implementation

.gitignore

2bfpbwt.c

Makefile

bf-pbwt.c

or topics

Contributors 3

Why is this important?

Why is this important?

- We can understand how to run a tool
- If we are developing competing method we can test it fairly
- If we are starting out we can get a grasp at how things work
- We can **replicate** and assume to get similar results
- …

What if we cannot reproduce the results?

What if we cannot reproduce the results?

We always do (and should) assume good faith

WIKIPEDIA
The Free Encyclopedia

List of scientific misconduct incidents

1 language

Article Talk Read Edit View history Tools

Scientific misconduct is the violation of the standard codes of scholarly conduct and ethical behavior in the publication of professional scientific research. A *Lancet* review on *Handling of Scientific Misconduct in Scandinavian countries* gave examples of policy definitions. In Denmark, scientific misconduct is defined as "intention[al] negligence leading to fabrication of the scientific message or a false credit or emphasis given to a scientist", and in Sweden as "intention[al] distortion of the research process by fabrication of data, text, hypothesis, or methods from another researcher's manuscript form or publication; or distortion of the research process in other ways."[1][2]

A 2009 systematic review and meta-analysis of survey data found that about 2% of scientists admitted to falsifying, fabricating, or modifying data at least once.[3]

Incidents should only be included in this list if the individuals or entities involved have their own Wikipedia articles, or in the absence of an article, where the misconduct incident is covered in multiple reliable sources.

How to do it with **snakemake**

Basic Syntax

```
rule RULE_NAME:
    input:
        "INPUT_FILE"
    output:
        "OUTPUT_FILE"
    shell:
        '''
            Command to execute using input and output
        '''
```

Basic Syntax

```
rule cat:
    input:
        "test.fa"
    output:
        "catted.fa"
    shell:
        '''
            cat {input} > {output}
        '''
```

Example rules

```
rule cat:
    input:
        f_in="test.fa"
    output:
        f_out="catted.fa"
    shell:
        '''
            cat {input.f_in} > {output.f_out}
        '''
```

Example rules

```
rule cat:
    input:
        f_in1="fa1.fa",
        f_in2="fa2.fa",
    output:
        f_out="catted.fa"
    shell:
        '''
            cat {input.f_in1} {input.f_in2} > {output.f_out}
        '''
```

Example rules

```
rule wc:
    input:
        "catted.fa"
    output:
        count=wc.txt
    shell:
        '''
            wc {input} > {output.count}
        '''
```

Example rules

```
rule grep:
    input:
        catted="catted.fa",
    output:
        s="seqs.txt",
        a="a.txt",
        c="c.txt",
    shell:
        '''
        grep "^>" {input.catted} -c > {output.g}
        grep "[Aa]" {input.catted} -c > {output.a}
        grep "[Cc]" {input.catted} -c > {output.c}
        '''
```

How to run the workflow

Save the rules in one file called “Snakefile”

```
snakemake
```

Save to a file called “test.smk”

```
snakemake -s test.smk
```

More realistic example

```
rule index_fa:
    input:
        "hs37d5.fa"
    output:
        "hs37d5.fa.fai"
    shell:
        '''
        samtools faidx {input}
        '''
```

```
rule get_region:
    input:
        fa="hs37d5.fa",
        fai="hs37d5.fa.fai",
    output:
        "sub.fa"
    params:
        reg="21:34407410-40062514",
    shell:
        '''
        samtools faidx {input.fa} {params.reg} > {output}
        '''
```

More realistic example

```
rule index_fa:
    input:
        "hs37d5.fa"
    output:
        "hs37d5.fa.fai"
    shell:
        '''
        samtools faidx {input}
        '''
```

```
rule get_region:
    input:
        fa="hs37d5.fa",
        fai="hs37d5.fa.fai",
    output:
        "sub.fa"
    params:
        reg="21:34407410-40062514",
    shell:
        '''
        samtools faidx {input.fa} {params.reg} > {output}
        '''
```

More realistic example

```
rule index_fa:
    input:
        "hs37d5.fa"
    output:
        "hs37d5.fa.fai"
    shell:
        '''
        samtools faidx {input}
        '''
```

```
rule get_region:
    input:
        fa="hs37d5.fa",
        fai="hs37d5.fa.fai",
    output:
        "sub.fa"
    params:
        reg="21:34407410-40062514",
    shell:
        '''
        samtools faidx {input.fa} {params.reg} > {output}
        '''
```

snakemake -s test.smk -c1

More realistic example

```
rule index_fa:
    input:
        "hs37d5.fa"
    output:
        "hs37d5.fa.fai"
    shell:
        '''
        samtools faidx {input}
        '''
```

```
rule get_region:
    input:
        fa="hs37d5.fa",
        fai="hs37d5.fa.fai",
    output:
        "sub.fa"
    params:
        reg="21:34407410-40062514",
    shell:
        '''
        samtools faidx {input.fa} {params.reg} > {output}
        '''
```

snakemake -s test.smk -c1 -f get_region

Rle *run* at the beginning

```
rule run:
    input:
        "sub.fa"
```

snakemake -s test.smk -c1

DAG

snakemake -s test.smk --dag

```
digraph snakemake_dag {
    graph[bgcolor=white, margin=0];
    node[shape=box, style=rounded, fontname=sans, fontsize=10, penwidth=2];
    edge[penwidth=2, color=grey];
        0[label = "run", color = "0.22 0.6 0.85", style="rounded,dashed"];
        1[label = "get_region", color = "0.44 0.6 0.85", style="rounded,dashed"];
        2[label = "index_fa", color = "0.00 0.6 0.85", style="rounded,dashed"];
        1 -> 0
        2 -> 1
}
```

Expand

```
rule index_fa:
    input:
        "{sample}.fa"
    output:
        "{sample}.fa.fai"
    shell:
        '''
        samtools faidx {input}
        '''
```

Expand

```
rule get_region:
    input:
        fa="{sample}.fa",
        fai="{sample}.fa.fai",
    output:
        "{sample}.sub.fa"
    params:
        reg="21:100-200",
    shell:
        '''
        samtools faidx {input.fa} {params.reg} > {output}
        '''
```

Expand

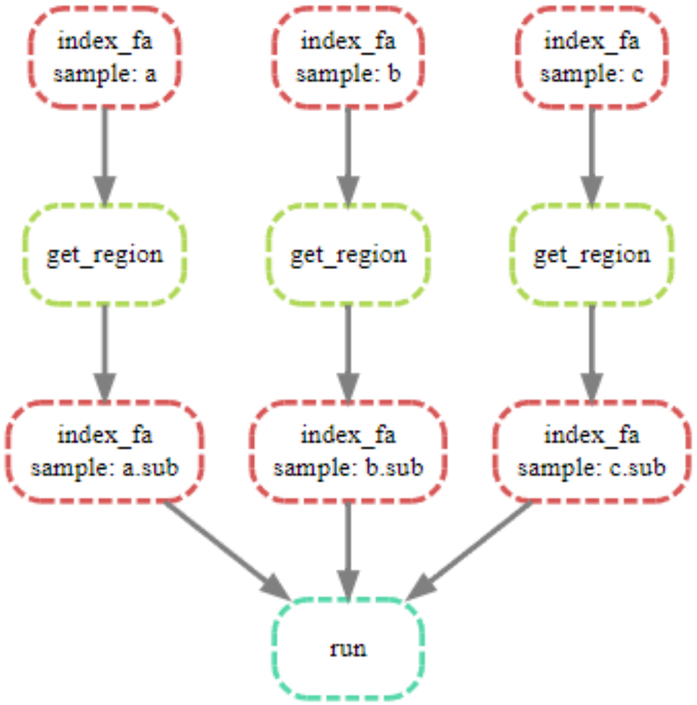
```
rule run:
    input:
        expand(
            "{sample}.sub.fa.fai",
            sample=["a", "b", "c"]
        )
```

Expand

```
rule run:
    input:
        expand(
            "{sample}.sub.fa.fai",
            sample=["a", "b", "c"]
        )
```

Expand

```
rule run:
    input:
        expand(
            "{sample}.sub.fa.fai",
            sample=["a", "b", "c"]
        )
```



The expand function - 2

```
rule index_fa:
    input:
        "{ref}.fa",
    output:
        "{ref}.fa.fai",
    shell:
        '''
        samtools faidx {input}
        '''
```

The expand function - 2

```
rule get_region:
    input:
        fa="{ref}.fa",
        fai="{ref}.fa.fai",
    output:
        "{ref}.sub-{region}.fa"
    shell:
        '''
        samtools faidx {input.fa} {wildcards.region} > {output}
        '''
```

The expand function - 2

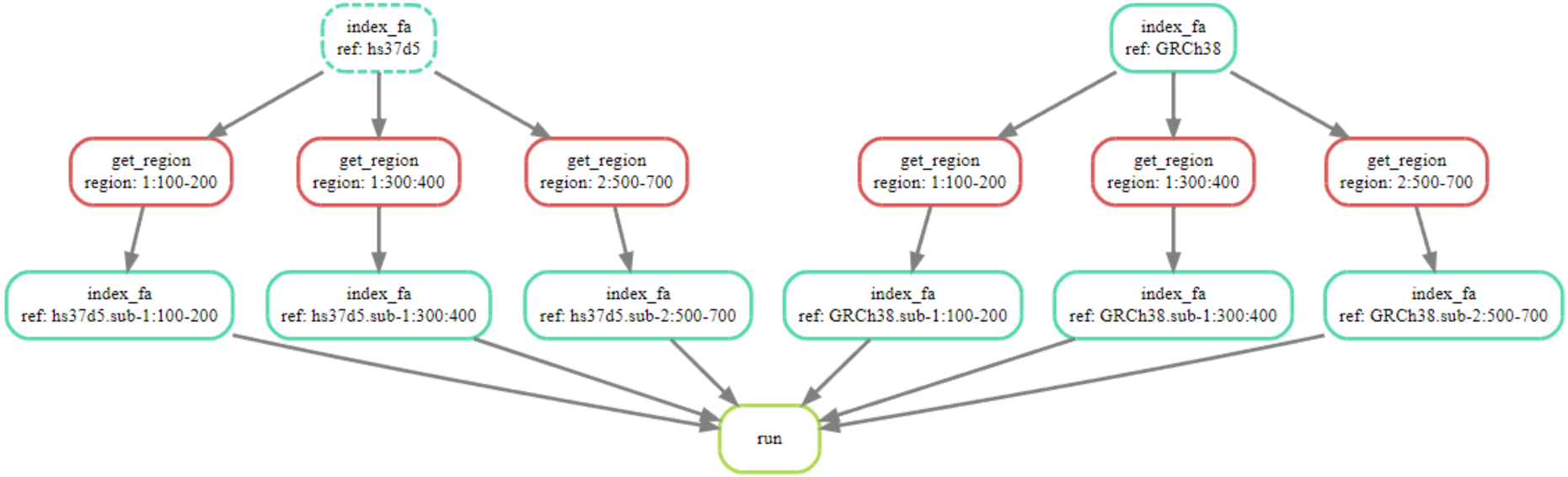
```
rule run:
    input:
        expand(
            "{ref}.sub-{region}.fa.fai",
                ref=[“hs37d5”, “GRCh38”],
            region=["1:100-200”, “1:300:400”, “2:500-700”],
        )
```

The expand function - 2

```
rule run:
    input:
        expand(
            "{ref}.sub-{region}.fa.fai",
                ref=["hs37d5", "GRCh38"],
            region=["1:100-200", "1:300:400", "2:500-700"],
        )
```

The expand function - 2

```
rule run:
    input:
        expand(
            "{ref}.sub-{region}.fa.fai",
                ref=[“hs37d5”, “GRCh38”],
            region=["1:100-200”, “1:300:400”, “2:500-700”],
        )
```



Hands-on session

https://github.com/sciccolella/snakemake_lab